



Engenharia Informática
Introduction to Machine Learning

Deliverable 2

Projeto IAA

Predictive Monitoring of Cloud Cluster Resources
Using Time Series Models

Authors:

Diego Aguilar - 117122
Dinis Oliveira - 119193
Diogo Pinheiro - 119907

This project is conducted by three students enrolled in the same practical class, adhering to the required group composition.

24/04/2026

1 Context and Motivation

Modern cloud clusters host thousands of machines running heterogeneous workloads under strict Service Level Agreements. When resource utilization spikes unexpectedly, operators must react quickly to prevent cascading failures. In practice, this diagnosis still relies on manual inspection of dashboards, a process that is slow, unscalable, and inherently reactive.

A more effective approach is to **predict** resource utilization before it reaches critical levels. If a model can forecast, for instance, that a machine’s CPU will exceed 90% within the next 15 minutes, operators gain a window to take preventive action such as migrating workloads or scaling resources.

We frame this as a **time series forecasting problem**: given the recent history of a machine’s metrics (CPU, memory, disk, network), predict the future value of those metrics at a configurable horizon. This formulation is naturally supervised, since the target is the actual future observation recorded by the monitoring system, and it explicitly respects the temporal structure of the data.

2 Dataset Description

2.1 Source and Scale

We use the **Alibaba Cluster Trace v2018**, a publicly available dataset containing production telemetry from a cluster of approximately 4000 machines over 8 days. The data was collected from Alibaba’s Fuxi scheduler and published for research purposes.

- Repository: <https://github.com/alibaba/clusterdata/tree/master/cluster-trace-v2018>
- Scale: ~247 million rows across all machines.
- Sampling interval: approximately 300 seconds (5 minutes).

This dataset was chosen over alternatives such as the Numenta Anomaly Benchmark (NAB), which we initially considered in Deliverable 1. Unlike NAB, which provides univariate series without multi-resource context, the Alibaba trace offers multivariate telemetry across CPU, memory, network, and disk for each machine, enabling the model to learn cross-resource dependencies.

2.2 Primary Table: `machine_usage`

The main table used in this project contains 9 columns recorded at regular intervals for each machine. Table 1 describes the schema.

2.3 Supporting Table: `machine_meta`

Static metadata describing each machine’s hardware configuration is available in `machine_meta` (Table 2). This table enables contextual analysis, such as correlating forecasting difficulty with machine capacity or failure domain.

2.4 Machine Selection and Profiling

The full dataset contains ~4000 machines, but not all are equally informative for training forecasting models. We implemented a two-pass profiling and selection pipeline:

Table 1: Schema of `machine_usage` (9 columns, no header in raw CSV).

Column	Type	Description
<code>machine_id</code>	string	Unique identifier of the machine
<code>time_stamp</code>	double	Time offset in seconds from the start of the trace
<code>cpu_util_percent</code>	int	CPU utilization $[0, 100]$
<code>mem_util_percent</code>	int	Memory utilization $[0, 100]$
<code>mem_gps</code>	double	Normalized memory bandwidth $[0, 100]$
<code>mkpi</code>	int	Cache misses per thousand instructions
<code>net_in</code>	double	Normalized incoming network traffic $[0, 100]$
<code>net_out</code>	double	Normalized outgoing network traffic $[0, 100]$
<code>disk_io_percent</code>	double	Disk I/O utilization $[0, 100]$; values of -1 or 101 denote invalid readings

Table 2: Schema of `machine_meta` (7 columns).

Column	Type	Description
<code>machine_id</code>	string	Unique identifier
<code>time_stamp</code>	bigint	Timestamp in seconds
<code>failure_domain_1</code>	bigint	First level of failure domain (enumerated)
<code>failure_domain_2</code>	string	Second level of failure domain
<code>cpu_num</code>	bigint	Number of CPU cores
<code>mem_size</code>	bigint	Normalized memory size $[0, 100]$
<code>status</code>	string	Machine status (online/offline)

1. **Profiling pass:** A streaming scan over all 247M rows computes per-machine statistics without loading the full dataset into memory. For each machine, we compute: mean, standard deviation, and percentiles (p5, p25, p50, p75, p95, p99) of CPU utilization; percentage of time spent in idle ($< 5\%$), low, medium, and high utilization bands; burstiness (p95/p50 ratio and spike count, defined as transitions from $< 30\%$ to $> 70\%$ CPU); temporal trend via linear regression slope; and gap analysis (maximum gap between consecutive measurements).
2. **Behavioral classification:** Each machine is assigned a behavioral profile based on these statistics: *stable_high*, *stable_mid*, *stable_low* (consistent load at different levels), *bursty* (frequent spikes), *idle_with_spikes* (mostly idle with occasional activity), *trending* (significant upward or downward trend), and *erratic* (high variance without clear pattern).
3. **Stratified sampling:** Rather than selecting only the “best” machines by a single metric, we sample across all behavioral profiles to ensure diversity. The final dataset of 100 machines includes representation from each profile, weighted toward profiles that are most informative for forecasting (e.g., higher proportion of bursty and *stable_high* machines, while still including idle and trending machines).

This approach ensures the model encounters the full range of behaviors present in production, rather than being trained exclusively on high-activity machines that would

bias its predictions.

2.5 Limitations and Potential Bias

- **Single-cluster origin:** All data comes from one Alibaba cluster. Utilization patterns, scheduling policies, and hardware configurations may not generalize to other cloud providers or on-premises deployments.
- **Limited temporal span:** The 8-day trace may not capture weekly or monthly seasonality patterns.
- **Normalized metrics:** All values are normalized to $[0, 100]$, meaning absolute resource capacities are not available.
- **Historical data:** The trace was collected in 2018; workload characteristics and hardware may have evolved.
- **Predominantly idle cluster:** The majority of machines exhibit low utilization most of the time, reflecting real datacenter behavior but creating a skewed distribution that the models must handle.

3 Data Processing Methodology

3.1 Cleaning and Validation

Raw data requires several cleaning steps before it can be used for model training:

- **Invalid values:** The documentation specifies that `disk_io_percent` values of -1 or 101 are invalid sensor readings; these are replaced with NaN.
- **Range enforcement:** All percentage metrics are clipped to $[0, 100]$. Values outside this range in other columns are also treated as NaN.
- **Missing data:** Short gaps (< 5 consecutive missing values) are filled via linear interpolation within each machine’s series. Longer gaps are not interpolated to avoid fabricating data; instead, the series is segmented into continuous sub-series at gap boundaries.

3.2 Sliding Window Construction

To transform the time series into a supervised learning format, we apply a sliding window approach. For a window size W and a forecast horizon H :

$$\mathbf{X}_t = \begin{bmatrix} x_{t-W+1} \\ x_{t-W+2} \\ \vdots \\ x_t \end{bmatrix} \in R^{W \times 7}, \quad y_t = \text{cpu.util}(t + H) \quad (1)$$

where each row x_i contains the 7 metric values at timestamp i , and y_t is the target CPU utilization H steps into the future. This formulation naturally provides supervised labels from the data itself, without requiring external annotation.

For the XGBoost model, the window matrix $\mathbf{X}_t$ is flattened into a feature vector augmented with statistical aggregates: mean, standard deviation, minimum, maximum, and linear slope of each metric within the window.

For the LSTM model, the window matrix $\mathbf{X}_t$ is fed directly as a sequence, preserving temporal ordering.

3.3 Chronological Split

Because the data is a time series, random shuffling would allow the model to “see” future observations during training, leading to artificially inflated performance that does not reflect real deployment. We enforce a strict chronological split:

- **Training set:** First 60% of each machine’s timeline.
- **Validation set:** Next 20%, used for hyperparameter tuning.
- **Test set:** Final 20%, used only for final evaluation.

The normalization scaler (min-max to $[0, 1]$) is fitted exclusively on the training set and applied to validation and test sets. Sliding windows do not span across split boundaries.

4 Problem Formulation

We define the forecasting problem in two complementary formulations that share the same input structure but differ in the output:

4.1 Regression: Predicting Future Metric Values

- **Input X:** A temporal window of the last W observations across 7 resource metrics.
- **Output y :** The continuous value of the target metric (e.g., CPU utilization) at time $t + H$.
- **Evaluation:** Mean Absolute Error (MAE), Root Mean Square Error (RMSE), and R^2 coefficient.

This formulation provides the most information, as the model must estimate the magnitude of future utilization, enabling graded responses (e.g., “CPU will be at 78%” vs. “CPU will be at 95%”).

4.2 Classification: Predicting Threshold Violations

- **Input X:** Same temporal window as above.
- **Output y :** Binary label indicating whether the target metric will exceed a predefined threshold τ at time $t + H$ (e.g., $\tau = 80\%$ for CPU).
- **Evaluation:** F1-Score (primary, due to class imbalance), Precision, Recall, AUC-ROC, and Confusion Matrix. We deliberately avoid relying on Accuracy, as the low frequency of threshold violations makes it a misleading metric.

We acknowledge that the threshold τ is a design choice. However, unlike the label engineering approach proposed in Deliverable 1 (where thresholds defined the class identity), here the underlying target is the real observed value. The threshold merely discretizes a genuine continuous measurement.

4.3 Forecast Horizons

We will evaluate multiple horizons H to characterize how prediction accuracy degrades with distance: 5 minutes (1 step), 15 minutes (3 steps), 30 minutes (6 steps), and 1 hour (12 steps). This analysis informs operators about the practical lead time the model can provide.

5 Models Explored and Justification

We selected three models that represent different points on the complexity–interpretability spectrum. All three receive the same input data and are evaluated on the same test set, enabling direct comparison.

5.1 Baseline: Linear Regression / Logistic Regression

A simple linear model serves as the minimum-viable baseline. If the more complex models do not significantly outperform this baseline, it would indicate either that the forecasting task is inherently simple (and a complex model is unnecessary) or that the feature engineering is inadequate.

For the regression task, we use ordinary linear regression on the aggregated features. For the classification task, logistic regression with the same features. This baseline is computationally trivial and fully interpretable.

5.2 XGBoost: Gradient Boosted Trees

XGBoost is selected as the primary non-sequential model for its established performance on tabular data and its built-in support for feature importance analysis (Chen & Guestrin, 2016).

- **Input:** Statistical features extracted from the sliding window (mean, standard deviation, minimum, maximum, and linear slope of each metric), producing a fixed-length feature vector per sample.
- **Strengths:** Fast training, robust to missing values, interpretable via SHAP or feature importance scores. The feature importance analysis will reveal which metrics and which temporal statistics are most predictive of future utilization.
- **Hyperparameters:** We plan to tune learning rate, maximum tree depth, number of estimators, and subsampling ratio via grid search on the validation set.

5.3 LSTM: Long Short-Term Memory Network

To capture temporal dependencies that may be lost during feature aggregation, we implement an LSTM architecture (Hochreiter & Schmidhuber, 1997).

- **Input:** The raw sequence of W timestamps $\times$ 7 metrics, preserving temporal ordering. No manual feature engineering is required.
- **Strengths:** Can learn complex, non-linear temporal patterns such as delayed cross-metric effects (e.g., a network traffic spike that precedes a CPU increase with a lag). The gating mechanism addresses the vanishing gradient problem for longer windows.
- **Architecture:** We plan to start with a single LSTM layer followed by a dense output layer, increasing complexity only if validation performance justifies it.

5.4 Model Comparison Strategy

The comparison between XGBoost and LSTM addresses a fundamental question: *does preserving raw temporal order provide an advantage over hand-crafted statistical summaries?* The baseline anchors this comparison by establishing the performance floor.

Table 3 summarizes the evaluation metrics for each formulation.

Table 3: Evaluation metrics by task formulation.

Metric	Regression	Classification
MAE / RMSE	Primary	—
R^2	Secondary	—
F1-Score (macro)	—	Primary
Precision / Recall	—	Secondary
AUC-ROC	—	Secondary
Confusion Matrix	—	Reported

6 Preliminary Ethical Considerations

6.1 Privacy and Anonymization

The Alibaba Cluster Trace is published with anonymized identifiers: machine IDs and job names are opaque hashes that cannot be linked to specific users, applications, or customers. The data is from 2018 and has been publicly available for research since its release.

6.2 Bias and Representativeness

The dataset reflects the operational patterns of a single Alibaba cluster. Conclusions drawn from this data should not be assumed to generalize to other providers (AWS, Azure), different hardware generations, or on-premises environments. The predominance of idle machines in the cluster is a genuine characteristic of production datacenters, but it means the model will be disproportionately exposed to low-utilization patterns.

6.3 Impact of Prediction Errors

In an operational deployment, the consequences of prediction errors are asymmetric. A **false negative** (failing to predict a spike) means the operator receives no warning, equivalent to the current manual monitoring baseline. A **false positive** (predicting a spike that does not occur) could trigger unnecessary interventions, waste resources, or cause alert fatigue. The trade-off between precision and recall should be calibrated to the criticality of the system: in safety-critical environments, higher recall (fewer missed spikes) is preferable even at the cost of more false alarms.

6.4 Responsible Use

The forecasting model is intended as a decision-support tool, not a replacement for human judgment. It should not be used to evaluate the performance of individuals or teams. Any automated actions triggered by the model’s predictions should include human oversight and override mechanisms.

6.5 Reproducibility

The dataset is public and freely accessible. All code, preprocessing pipelines, hyperparameters, and random seeds will be documented and made available to ensure full

reproducibility of our results.

7 Adjustments from Deliverable 1

Following the feedback received on Deliverable 1 and our own reassessment, the project has undergone several significant changes:

- **From classification to forecasting:** Deliverable 1 framed the problem as multi-class classification of system states (CPU-bound, Memory-bound, etc.) using threshold-derived labels. We now formulate it as time series forecasting, where the target is the actual future value recorded by the monitoring system. This eliminates the subjectivity of threshold-based label engineering and makes the problem genuinely supervised.
- **New dataset:** We replaced the Numenta Anomaly Benchmark (NAB) with the Alibaba Cluster Trace v2018. NAB provides univariate series without multi-resource context, while the Alibaba trace offers multivariate telemetry (7 metrics per machine) from a production environment with approximately 4 000 machines over 8 days.
- **Temporal awareness:** Deliverable 1 did not adequately address the sequential nature of the data. We now enforce chronological data splits, use sliding windows that respect temporal ordering, and include an LSTM architecture designed for sequential input alongside the tabular XGBoost approach.
- **Predictive orientation:** Rather than classifying the current state of a machine, the models now predict future metric values at configurable horizons (5 min to 1 hour ahead), providing actionable lead time for operators.
- **Baseline model:** We added a linear regression/logistic regression baseline to anchor the comparison and verify that model complexity is justified.
- **Expanded evaluation:** The evaluation framework now covers both regression metrics (MAE, RMSE, R^2) and classification metrics (F1-Score, AUC-ROC), reflecting the dual formulation of the problem.

8 Acknowledgment of AI Tools

In accordance with the project guidelines, we acknowledge the use of the following AI tools during the development of this deliverable:

- **Claude (Anthropic):** Used for exploratory data analysis of the Alibaba dataset, development of the machine profiling and selection pipeline, and synthesis of alternative problem formulations during the project redesign.
- **Gemini (Google):** Used to assist with structural organization and formatting of this document.

All technical decisions, including the problem formulation, model selection, data processing methodology, and evaluation strategy, were made by the group. The code for model training and evaluation will be developed and validated by the authors.